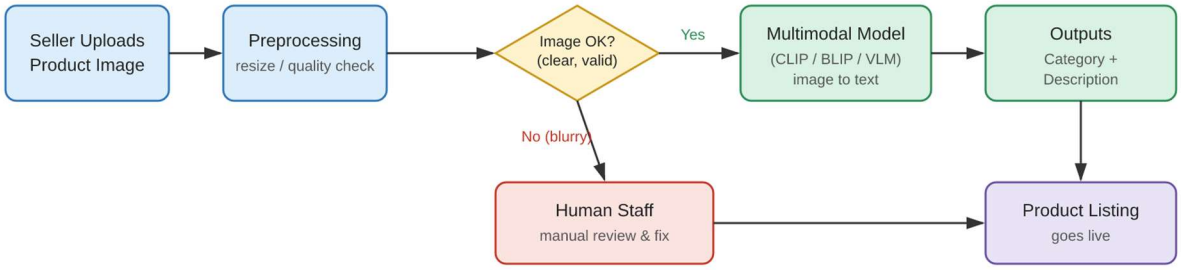
Product Understanding System Architecture

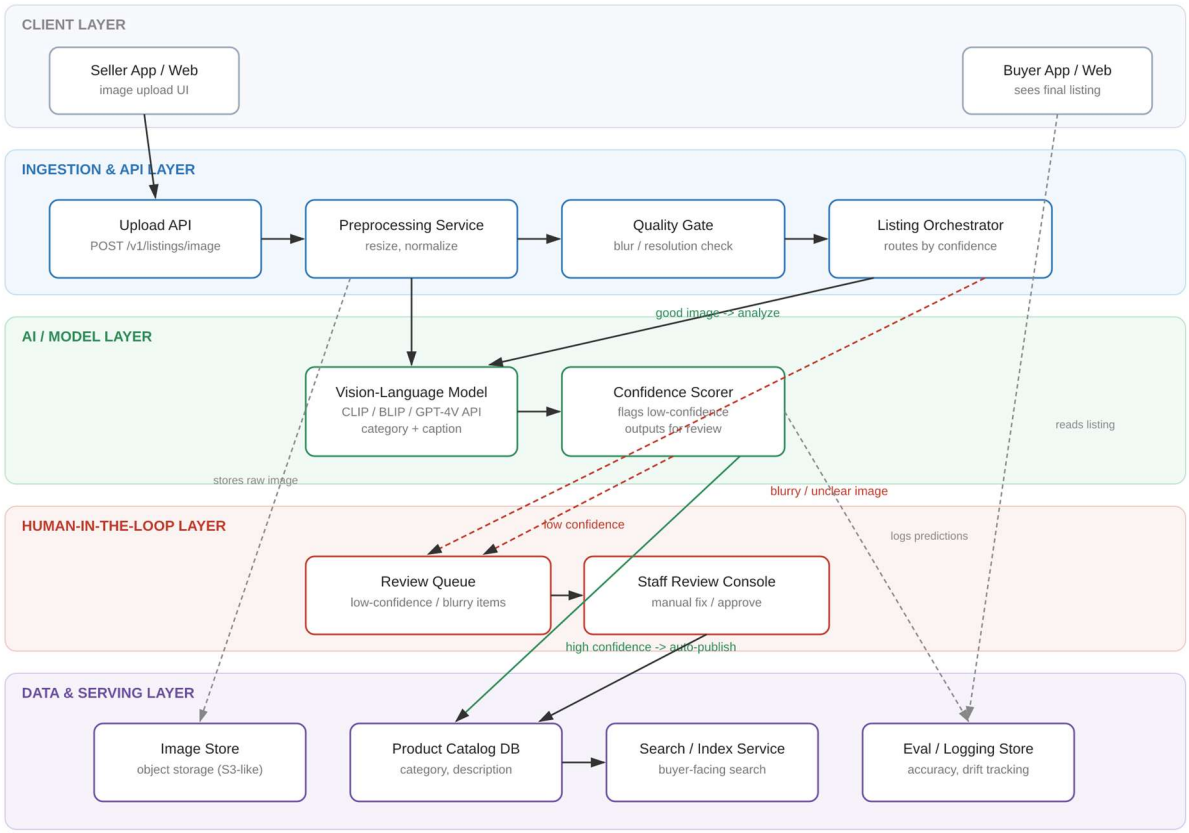
How the pipeline works:

1. Image comes in. The seller uploads a product photo at the time of listing.
2. Quality check. The image is resized, normalized, and screened for basic quality issues — blur, poor lighting, the product not actually being in frame.
3. Clean images go to the model. If the image passes the check, it's sent to the vision-language model, which returns a category and a draft description.
4. Bad images go to a person. If the image fails the check, it's routed to a human reviewer instead of being auto-published — this is the safety net, and it matters more than it sounds (more on this below).
5. The listing goes live. Whether the category and description came from the model or from a human fix, the final result is saved and the listing is published.



Goal: given a seller's product image, automatically produce a category and description, with a built-in fallback to human review when the model can't be trusted on a given image. Below is the architecture — components, data flow, and where each design decision (especially the human handoff) sits in the system.

System architecture ::



Client Layer

- Seller App / Web — where the seller uploads the product photo.
- Buyer App / Web — reads the finished listing once it's published; not part of the AI flow.

Ingestion & API Layer

- Upload API — single entry point for new images (POST /v1/listings/image).
- Preprocessing Service — resizes and normalizes the image so the model gets consistent input.
- Quality Gate — rejects images that are blurry, too low-resolution, or don't clearly show a product, before any model call is wasted on them.
- Listing Orchestrator — the controller that decides where an image goes next: forward for AI analysis, or straight to human review if it failed the quality gate.

AI / Model Layer

- Vision-Language Model — CLIP / BLIP, or a single modern VLM (GPT-4V, Gemini Vision, LLaVA) — produces the category and a draft description from the image.
- Confidence Scorer — checks how sure the model is about its own output. Low-confidence predictions get flagged for review instead of being trusted blindly.

Human-in-the-Loop Layer

- Review Queue — holds anything the system isn't confident about: failed quality-gate images and low-confidence model outputs.
- Staff Review Console — where a human looks at the flagged item and corrects or approves it before it's allowed to go live.

Data & Serving Layer

- Image Store — raw uploaded images, kept for audit and re-processing.
- Product Catalog DB — the source of truth: final category + description, whether it came from the model or a human fix.
- Search / Index Service — what buyers actually query against.
- Eval / Logging Store — logs every prediction and confidence score, used later to measure accuracy and catch model drift over time.

Why the Human-in-the-Loop Layer Exists

Sir mentioned this in class about Amazon, and it shaped this design: the automation doesn't run unsupervised. When a photo is blurry or the model isn't confident, the listing is routed to a person instead of being auto-published. That's why this architecture has two separate trigger paths into the Review Queue — one from the Quality Gate (bad image, never even reaches the model) and one from the Confidence Scorer (image was fine, but the model's answer wasn't trustworthy). Both paths exist on purpose; a single point of failure here would mean wrong listings going live.

Evaluation & Business Fit::

- Model accuracy is tracked from the Eval / Logging Store — precision/recall on category, and periodic human spot-checks on description quality (automated text metrics like BLEU/ROUGE don't catch a confidently wrong description).
- Business impact is tracked downstream of the architecture: listing turnaround time, and buyer click-through/conversion once descriptions are consistent.
- This matters because it replaces slow, inconsistent manual tagging at scale, while the Human-in-the-Loop Layer keeps a bad photo from turning into a bad listing.

Github.com/Devpathak18